
2. Erarbeitungs-/Reflexionsphase

Objektorientierte und funktionale Programmierung mit Python

DLBDSOOFPP01_D

Dennis Simon Lätsch (IU14096102)

Softwareentwicklung (B.SC.)

15. Juni 2026

Tutor: Dr. Aiko Walte

Abbildungsverzeichnis

1	Angepasstes UML-Klassendiagramm nach der Reflexion	2
2	UML-Klassendiagramm der Gesamtarchitektur	4
3	Json-Struktur des OO-Modells	6

1 Untersuchung der Umsetzung objektorientierter Konzepte in Python

In der Erarbeitungs- und Reflexionsphase wurde untersucht, wie sich das in Phase 1 entwickelte objektorientierte Modell in Python umsetzen lässt. Dafür wurde das Modell in einem kleinen Testprogramm erprobt, in dem Module zu einem Studium hinzugefügt und nachträglich Noten eingetragen werden können. Im Mittelpunkt standen Klassen, Enumerationen, Komposition, optionale Beziehungen, Properties sowie die Frage, ob das ursprüngliche Klassendiagramm aus der Konzeptionsphase unverändert übernommen werden kann.

Die fachlichen Klassen `Studium`, `Modul` und `Pruefungsleistung` lassen sich gut in Python abbilden. Da diese Klassen überwiegend Daten speichern, wurden sie als `dataclasses` umgesetzt. Dadurch erzeugt Python Standardfunktionen wie den Konstruktor automatisch, wodurch die Klassen kompakt und gut lesbar bleiben. Für feste Wertebereiche wie den Modulstatus und die Prüfungsform wurden `Enum`-Klassen verwendet. Dadurch können nur definierte Werte wie `OFFEN`, `IN_ARBEIT`, `FERTIG`, `ANERKANNT` oder `KLAUSUR` verwendet werden. Dies verhindert uneinheitliche Freitexteingaben und vereinfacht spätere Auswertungen.

Ein wichtiger Umsetzungsaspekt war die Komposition. Die Beziehung zwischen `Studium` und `Modul` wird in Python durch eine Liste umgesetzt. Die Klasse `Studium` enthält daher ein Attribut `module: list[Modul]`. Bei `dataclasses` wird hierfür `field(default_factory=list)` verwendet. Dies ist notwendig, damit jedes `Studium`-Objekt eine eigene Modulliste erhält. Das Testprogramm bestätigte, dass Module auf diese Weise dynamisch zu einem Studium hinzugefügt werden können.

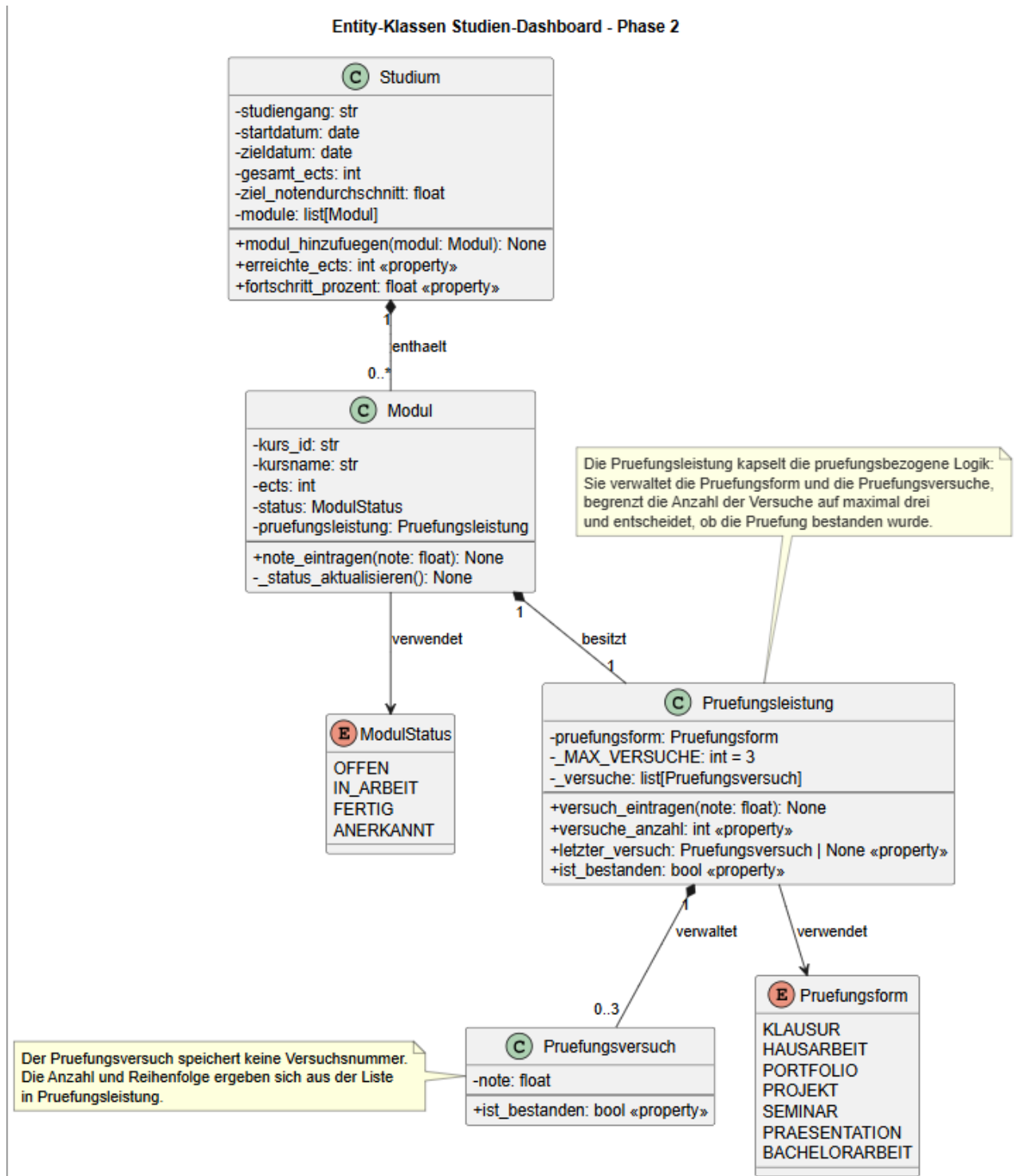
Während der Implementierung zeigte sich, dass das ursprüngliche Modell für Prüfungsleistungen zu einfach war. Im ersten Entwurf enthielt die Klasse `Pruefungsleistung` lediglich eine Note. Das Testprogramm zeigte jedoch, dass auch nicht bestandene Prüfungsversuche berücksichtigt werden müssen, da vor dem letzten Versuch in der Regel maximal zwei Fehlversuche möglich sind. Zunächst wurde diese Logik in der Klasse `Modul` umgesetzt. Dazu wurden ein interner Zähler für die Versuche und eine Konstante für die maximale Anzahl der Versuche ergänzt.

Diese Lösung funktionierte technisch, war objektorientiert jedoch nicht überzeugend. Die Klasse `Modul` enthielt dadurch zu viel prüfungsbezogene Logik. Außerdem konnte bei mehreren Versuchen immer nur die letzte Note gespeichert werden. Frühere Prüfungsversuche wären verloren gegangen und damit nicht mehr nachvollziehbar gewesen. Eine erste Alternative war, Prüfungsleistungen direkt als Liste im Modul zu speichern. Dadurch hätten zwar mehrere Versuche abgelegt werden können, die Validierung der maximalen Versuchszahl und die Bestehenslogik wären jedoch weiterhin in der Klasse `Modul` geblieben.

Aus diesem Grund wurde das Modell angepasst. Die prüfungsbezogene Logik wurde in die Klasse `Pruefungsleistung` verschoben. Diese verwaltet nun eine Liste von `Pruefungsversuch-`

Objekten. Ein `Pruefungsversuch` speichert genau eine `Note`, während die `Pruefungsleistung` die Versuchszählung, die Begrenzung auf maximal drei Versuche und die Entscheidung über das Bestehen kapselt. Außerdem wurde die Prüfungsform in die Klasse `Pruefungsleistung` verschoben, da sie fachlich enger zur Prüfung als zum Modul gehört.

Abb. 1. Angepasstes UML-Klassendiagramm nach der Reflexion



Quelle: Eigene Darstellung.

Eine Konsequenz dieser Modellierung ist, dass ein Modul immer genau eine `Pruefungsleistung`

besitzt. Diese speichert mindestens die Prüfungsform und kann zunächst ohne Prüfungsversuche existieren. Eine noch nicht abgelegte Prüfung wird daher nicht mehr durch eine fehlende Prüfungsleistung modelliert, sondern durch eine vorhandene `Pruefungsleistung` mit leerer Versuchsliste. Dadurch entfallen zusätzliche `None`-Prüfungen im Code. Die Klasse `Modul` bleibt schlanker und muss die Prüfungslogik nicht selbst verwalten. Das Testprogramm bestätigte, dass mit dieser Anpassung Module hinzugefügt und Prüfungsversuche nachträglich eingetragen werden können.

Für die Klasse `Pruefungsversuch` wurde `@dataclass(frozen=True)` verwendet. Dadurch ist ein gespeicherter Prüfungsversuch nachträglich nicht veränderbar. Diese Entscheidung passt fachlich, da ein dokumentierter Prüfungsversuch nicht versehentlich überschrieben werden sollte. Die Validierung der Note erfolgt in `__post_init__()`, weil diese Methode direkt nach der Objekterzeugung ausgeführt wird. So wird geprüft, ob eine eingegebene Note im zulässigen Bereich von 1,0 bis 6,0 liegt.

Ein weiteres Ergebnis betrifft berechnete Werte. Im UML-Modell war zunächst eine Methode `isBestandenePruefungsleistung()` vorgesehen. In der Python-Umsetzung wurde daraus eine `@property`, zum Beispiel `ist_bestanden`. Dadurch kann von außen wie auf ein Attribut zugegriffen werden, obwohl intern eine Berechnung erfolgt. Diese Lösung ist besser lesbar und entspricht eher dem üblichen Python-Stil. Dasselbe Prinzip kann später auch für Dashboard-Kennzahlen wie erreichte ECTS, Fortschritt in Prozent, Notendurchschnitt oder Velocity verwendet werden. Diese Werte werden nicht redundant gespeichert, sondern aus den vorhandenen Objektdaten berechnet.

Auf Vererbung wurde bewusst verzichtet. Eine mögliche Alternative wäre gewesen, verschiedene Arten von Prüfungsleistungen als Unterklassen zu modellieren. Für das aktuelle Dashboard bietet dies jedoch keinen fachlichen Mehrwert, da keine natürliche Generalisierungs- oder Spezialisierungsstruktur benötigt wird. Komposition bildet den Zusammenhang besser ab: Ein Studium enthält Module, ein Modul besitzt eine Prüfungsleistung, und eine Prüfungsleistung verwaltet mehrere Prüfungsversuche.

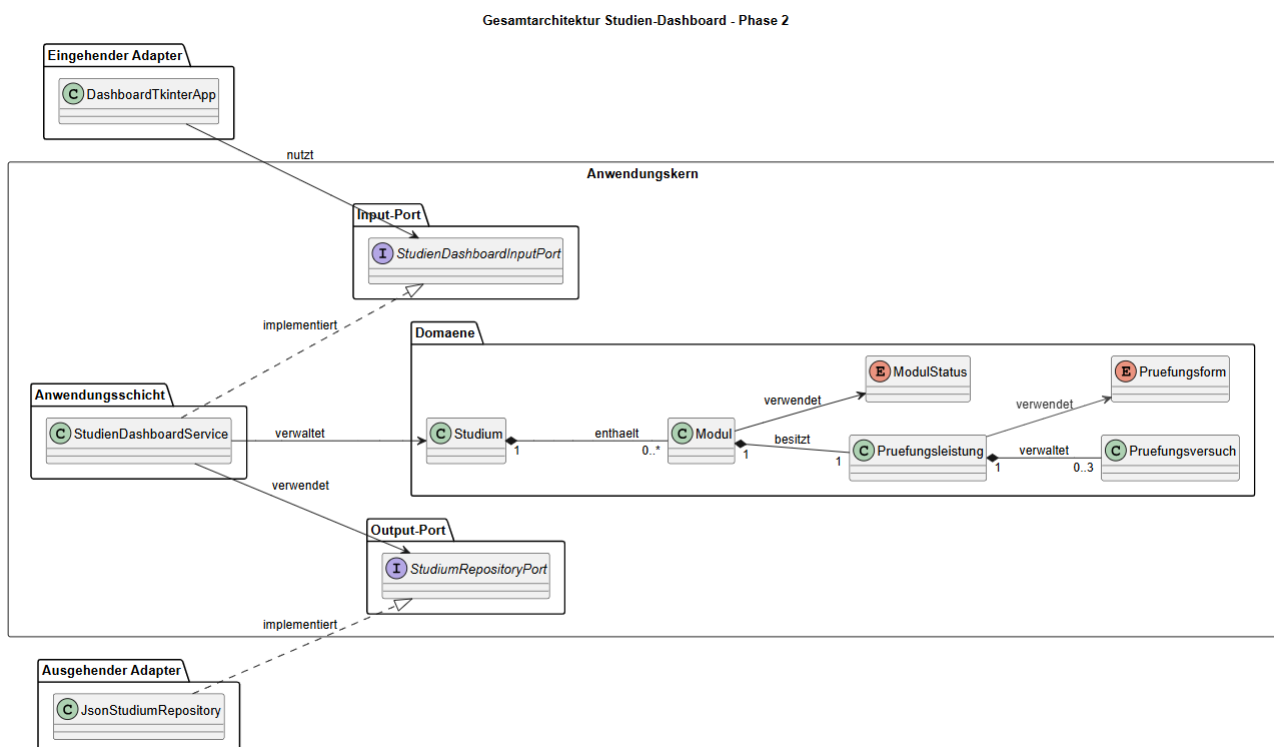
Das Klassendiagramm aus Phase 1 konnte damit fachlich weitgehend übernommen werden, musste jedoch an mehreren Stellen präzisiert werden. Neu hinzugekommen ist die Klasse `Pruefungsversuch`. Außerdem besitzt ein Modul nun genau eine Prüfungsleistung, die zunächst auch ohne Prüfungsversuche existieren kann. Die Prüfungsform wurde von `Modul` nach `Pruefungsleistung` verschoben. Zusätzlich wurden einzelne Details an den Python-Stil angepasst, zum Beispiel `snake_case` statt `CamelCase` und `@property` für berechnete Eigenschaften. Insgesamt zeigen die Testprogramme, dass das objektorientierte Modell mit Python gut umsetzbar ist. Die Anpassungen verbessern vor allem die Kapselung, die Lesbarkeit und die fachliche Trennung der Verantwortlichkeiten.

2 Gesamtarchitektur des Dashboard-Prototyps

Auf Grundlage der Untersuchung der objektorientierten Konzepte wurde die Gesamtarchitektur des späteren Python-Prototyps präzisiert. Ziel ist eine Architektur, bei der das Domänenmodell keine Abhängigkeiten nach außen besitzt. Die fachlichen Klassen sollen weder von der grafischen Oberfläche noch von der konkreten Datenspeicherung abhängen. Dadurch bleibt die Domäne gut testbar, wiederverwendbar und unabhängig von technischen Details. Änderungen an der Benutzeroberfläche oder am Repository sollen möglichst keine Anpassungen an den Entity-Klassen sowie deren Tests erforderlich machen.

Die Architektur orientiert sich vereinfacht an der hexagonalen Architektur, auch Ports-and-Adapters-Architektur genannt. Dieser Ansatz trennt den Anwendungskern von externen technischen Schnittstellen wie Benutzeroberflächen oder Datenbanken und legt die Abhängigkeitsrichtung nach innen fest: Adapter wie die `DashboardTkinterApp` und das `JsonStudiumRepository` hängen von den Ports des Anwendungskerns ab, während die Domäne selbst keine Abhängigkeiten zu diesen technischen Bestandteilen besitzt (Alister, 2005). Abbildung 2 zeigt die daraus abgeleitete Gesamtarchitektur des Prototyps.

Abb. 2. UML-Klassendiagramm der Gesamtarchitektur



Quelle: Eigene Darstellung.

Den Kern der Architektur bildet die Domäne. Sie enthält die fachlichen Klassen aus der Abbildung 1 Studium, Modul, Pruefungsleistung und Pruefungsversuch sowie die Enumerationen Modul-

`Status` und `Pruefungsform`. Diese Klassen repräsentieren den Studienverlauf und kapseln fachliche Regeln. Dazu gehören beispielsweise die Begrenzung der Prüfungsversuche, die Entscheidung, ob eine Prüfung bestanden wurde, sowie die Berechnung von Kennzahlen wie erreichte ECTS oder Fortschritt in Prozent. Die Domäne enthält keine Abhängigkeit zu Tkinter, JSON oder anderen technischen Bestandteilen.

Zwischen Domäne und Außenwelt wird eine Anwendungsschicht vorgesehen. Diese wird durch die Klasse `StudienDashboardService` abgebildet. Der Service stellt die zentralen Anwendungsfälle des Dashboards bereit, zum Beispiel das Hinzufügen eines Moduls, das Eintragen einer Note, das Laden und Speichern des Studiums sowie das Bereitstellen der Dashboard-Kennzahlen. Die Benutzeroberfläche greift dadurch nicht direkt auf die Domänenobjekte zu, sondern nutzt die vom Input-Port definierten Anwendungsfälle. Der `StudienDashboardService` implementiert diesen Input-Port und vermittelt zwischen Benutzeroberfläche, Domänenmodell und Persistenz. Sollte die Service-Klasse im weiteren Verlauf zu umfangreich werden, kann sie in mehrere spezialisierte Anwendungsklassen je nach Use-Case aufgeteilt werden.

Die Kommunikation mit dem Anwendungskern erfolgt über definierte Schnittstellen. Der `StudienDashboardInputPort` beschreibt die Anwendungsfälle, die von außen aufgerufen werden dürfen. Die Tkinter-Oberfläche `DashboardTkinterApp` nutzt diesen Input-Port und ist damit als eingehender Adapter modelliert. Sie ist für die Darstellung von Kennzahlen, Diagrammen und Modultabelle zuständig und nimmt Benutzereingaben entgegen. Fachliche Abläufe werden jedoch nicht direkt in der Oberfläche umgesetzt, sondern an den `StudienDashboardService` delegiert, der den Input-Port implementiert.

Für die Persistenz wird der `StudiumRepositoryPort` als Output-Port vorgesehen. Dieser beschreibt die Speicherfunktionen, die der Anwendungskern benötigt, insbesondere das Laden und Speichern eines `Studium`-Objekts. Das konkrete `JsonStudiumRepository` implementiert diesen Port als ausgehender Adapter. Der `StudienDashboardService` arbeitet dadurch nur gegen die Abstraktion und nicht gegen die konkrete JSON-Implementierung.

In Phase 1 war zunächst ein CSV-basiertes Repository geplant. Diese Entscheidung wurde nach der Reflexion angepasst. CSV eignet sich vor allem für flache, tabellarische Daten. Bereits das ursprüngliche Modell enthielt mit der Beziehung zwischen `Studium` und `Modul` eine verschachtelte Struktur, da ein `Studium` mehrere Module enthält. Durch die Erweiterung des Modells ist nun eine weitere Verschachtelung hinzugekommen: Jedes `Modul` besitzt eine `Pruefungsleistung`, und jede `Pruefungsleistung` verwaltet wiederum eine Liste von `Pruefungsversuch`-Objekten. Eine Speicherung in CSV wäre daher nur über mehrere Dateien und zusätzliche Zuordnungslogik sinnvoll umsetzbar. Dies würde die Persistenz gegenüber der ursprünglich geplanten Lösung deutlich komplexer machen.

Aus diesem Grund wird für den Prototyp ein `JsonStadiumRepository` vorgesehen. JSON kann verschachtelte Objekte und Listen direkt abbilden und gehört zur Python-Standardbibliothek dies wurde mit einem Kleinen Testprogramm, welches das OO-Modell einmal speichert und laden kann getestet (siehe Abbildung 3). Dadurch bleibt der Prototyp offline lauffähig und benötigt keine externen Bibliotheken. YAML wäre als lesbares Speicherformat ebenfalls denkbar, würde in Python jedoch typischerweise ein externes Paket erfordern. Da weiterhin keine externen Abhängigkeiten verwendet werden sollen, wird JSON gegenüber YAML bevorzugt.

Abb. 3. Json-Struktur des OO-Modells

```
1 {
2   "studiengang": "Software Engineering",
3   "startdatum": "2025-01-01",
4   "zieldatum": "2028-01-01",
5   "gesamt_ects": 10,
6   "ziel_notendurchschnitt": 2.0,
7   "module": [
8     {
9       "kurs_id": "k1",
10      "kursname": "Objektorientierte Programmierung mit Python",
11      "ects": 5,
12      "status": "FERTIG",
13      "pruefungsform": "PORTFOLIO",
14      "noten": [
15        5.0,
16        2.3
17      ]
18    },
19    {
20      "kurs_id": "k2",
21      "kursname": "Datenstrukturen und Algorithmen",
22      "ects": 5,
23      "status": "OFFEN",
24      "pruefungsform": "KLAUSUR",
25      "noten": []
26    }
27  ]
28 }
```

Quelle: Eigene Darstellung.

Die Außenwelt kommuniziert somit nicht direkt mit den Domänenklassen, sondern ausschließlich über definierte Schnittstellen mit dem Anwendungskern. Alles außerhalb des Anwendungskerns wird als Adapter betrachtet. Dadurch können die technischen Bestandteile außerhalb des Kerns gezielter ausgetauscht werden. So könnte die Tkinter-Oberfläche später durch eine andere Benutzeroberfläche oder das JSON-Repository durch ein echtes Datenbank-Repository ersetzt werden, ohne das Domänenmodell anpassen zu müssen. Dieser Austausch ist nicht automatisch aufwandsfrei, aber die notwendigen Änderungen bleiben stärker auf den jeweiligen Adapter begrenzt.

Das Ergebnis der Gesamtarchitektur ist ein mehrschichtiger Aufbau mit klar getrennten Verantwortlichkeiten. Das Domänenmodell bildet den Kern, die Anwendungsschicht stellt die Anwendungsfälle bereit, die Tkinter-Oberfläche dient als eingehender Adapter und das JSON-Repository als ausge-

hender Adapter für die Persistenz. Dieser Aufbau ist für einen kleinen Prototyp etwas aufwendiger als eine direkt gekoppelte Lösung, verbessert jedoch die Testbarkeit, die Wartbarkeit und die spätere Erweiterbarkeit des Systems.

Literaturverzeichnis

Alister, C. (2005). *Hexagonal-architecture*. The Hexagonal (Ports & Adapters) Architecture.

<https://alistair.cockburn.us/hexagonal-architecture>